

Predicting Protein–Ligand Interactions, Family, and Mutation Influence Using Multi-Modal Proxy Machine Learning

Tuba(Tori) Murphy, Hannah Fino, Paul Rubiro, Ana Barrera-Jauregui, Snigdha Chanduri

Due Date: May 12th, 2026

GitHub Link: <https://github.com/AT-1t/Chem-277B-MLP>

Abstract

This project presents an integrated computational framework for the prediction of small-molecule binding affinities in Epidermal Growth Factor Receptor (EGFR) kinase mutations associated with lung adenocarcinoma. EGFR plays an important role in cell growth and survival, and its mutations can often lead to abnormal activation and drug resistance, making accurate binding prediction essential for the development of targeted therapy. In order to tackle the limitations of other approaches that rely solely on protein sequence or chemical features, we combine biological, chemical, and mutation-specific data into a unified modeling pipeline. Our hypothesis is that a multi-stage fusion model that integrates kinase sequence features and ligand chemical properties with profiles of EGFR patient mutations can improve the accuracy of proxy predicted binding affinity for specialized therapeutics, creating a somewhat good framework for identifying therapeutic drug candidates for lung adenocarcinoma.

We used protein-ligand binding data from BindingDB for extracting sequence features, protein similarity measurements, and ligand chemical properties. Global alignment and local alignment methods were used to improve the feature-set, capturing both functional and structural similarities across kinase families. A fusion strategy is then applied to integrate patient-specific EGFR mutation data with ligand and protein features to enable binding predictions for personalized therapies. Machine learning models, including an artificial neural network for general binding affinity and a proxy LightGBM regression for patient specific predictions, are trained and evaluated with the LightGBM model achieving a predictive accuracy of 86%.

BindingDB Ligand Identifiers 37291, 37292, 37089, 37086, 37284, 37283, 618652, 37305, 27852, and 27853 were found to have repeatedly more favorable proxy binding affinity predictions across patient-specific EGFR mutation profiles. This implies that the proposed proxy binding model could act as a screening system to help identify ideal candidates for further chemotherapeutic scaffold improvements. Benzothienophene phenol analogs, thienopyrimidine kinase inhibitor analogs and genistein derivatives may present promising future directions in therapeutic development for EGFR-driven lung cancer.

1 Introduction

The Epidermal Growth Factor Receptor (EGFR) is a receptor tyrosine kinase essential for regulating key cellular processes including cell survival and division, being a key component in signaling pathways that control normal cellular function (J. Chen et al., 2016). Mutations in EGFR can lead to its continuous activation, causing uncontrolled cell division that drives the development of cancers, particularly lung adenocarcinoma, which is the most common type of non-small cell lung cancer. Due to its pivotal role in tumor progression, EGFR is an important target for anticancer therapies.

Targeted therapies, including tyrosine kinase inhibitors, have been developed to block EGFR activity. Despite having initial effectiveness, cancers often develop resistance to inhibitors (Koulouris et al., 2022). Gefitinib and Erlotinib are tyrosine kinase inhibitors that were developed to target abnormal EGFR signaling in patients with activating EGFR mutations. However, their long-term effectiveness is limited by the development of drug resistance (Kobayashi et al., 2005). This resistance is often caused by secondary mutations in the EGFR kinase domain that alter the structure of the protein and disrupt the drug binding process. The result is that the drug shows varying levels of effectiveness across different patients, highlighting the need for more precise and personalized treatment options at the patient level.

Protein-ligand binding affinity is an important indicator for understanding drug effectiveness. It measures how strongly a drug molecule interacts with its target protein. A high binding affinity often signifies a greater drug efficacy. The experimental determination of binding affinity, however, can be time-consuming and expensive, making computational prediction an attractive alternative. Databases such as BindingDB provide extensive protein-ligand interaction data, with protein sequences, ligand structures, and binding affinity values such as K_i , the inhibitor potency, K_d , the ligand binding

affinity, and IC50, the concentration needed to inhibit a response by 50% (Abdelkader and Kim, 2024). Using these extensive datasets, the development of machine learning models for predicting binding behavior can be explored.

Current progress in computational drug analysis has limitations. Some approaches focus primarily on protein sequence and structural information, while others might emphasize the chemical properties of ligands. Models that rely solely on sequence data may fail to capture the chemical complexity of ligand interactions, whereas models that focus only on ligand chemical features may be overlooking the structural and functional characteristics. This separation limits the ability of the previously mentioned models to represent the biological and chemical properties of protein-ligand binding. Additionally these limitations do not adequately incorporate mutation-specific information. Mutation-specific information is important for understanding cancer-related proteins such as EGFR where mutations are able to significantly affect binding sites, protein conformation, and drug-target interactions as a whole. Models may be developed at the general protein-level, but fail to extend models to mutation-specific information and patient-specific cases. This gap in adaptation restricts the ability to apply these models in targeted therapies, where treatment is dependent on an individual’s genetic profile (Hyman et al., 2017).

This project aims to address these limitations by proposing an integrated computational framework that combines the biological, chemical, and genetic mutation data into a unified modeling pipeline. BindingDB is used for gathering kinase-related protein-ligand interactions, focusing specifically on EGFR. Protein sequence features are extracted and then enhanced using global alignment and local alignment methods, such as Needleman-Wunsch and Smith-Waterman respectively. These enhancements allow for similarity and conserved functional motifs to be captured and then paired with ligand chemical features, such as molecular descriptors and Morgan fingerprints for encoding physicochemical properties. Features are further enhanced using sequence representations such as amino acid composition and deep sequence embeddings that help capture functional relationships within the protein sequences. The integration of these more diverse feature types allows for the models to capture more complex interactions between protein structure, ligand chemistry, and any binding behavior.

Building on the initial models, the pipeline extends to patient-specific modeling by using EGFR mutation data. The EGFR mutations in lung adenocarcinoma that were investigated include exon 19 deletions, L858R, L861Q and G719X, and exon 20 alterations. These mutations alter the structure and activity of the EGFR tyrosine kinase domain, resulting in abnormal signaling, increased ATP binding activity, and changes to ligand or inhibitor interactions. While some of the EGFR mutations can initially respond to tyrosine kinase inhibitors such as ones previously mentioned, gefitinib or erlotinib, others may cause resistance to therapies by modifying the kinase binding pocket and the structural conformation. Further understanding of how these mutations influence protein-ligand interactions is very important for improving precision medicine approach for EGFR driven cancer.

The experimental evaluation of protein-ligand interactions for every EGFR mutation can be time-consuming and costly, therefore computational proxy models provide a more efficient alternative for estimating biological behaviors. Proxy models are machine learning models that can be trained to approximate complex molecular interactions using existing biological, chemical, and experimental data. In this project artificial neural networks (ANN) are used to predict ligand binding affinity, which is then integrated with the XGBoost classifier into a gradient-boosting model, LightGBM, that is applied to perform patient-level predictions using proxy modeling.

2 Project Design

This is a multi-stage feature engineering pipeline where each checkpoint builds on the results of the previous one. The output of one step becomes the input of the next, expanding from molecular features to chemical context to patient level prediction, with sanity checks at every step before moving forward. Checkpoint 1 has four steps. It downloads BindingDB kinase and ligand data, builds Needleman–Wunsch global sequence alignment features, and combines the protein and chemical features into a singular dataset. Finally, it trains a neural network regression model to calculate EGFR binding affinity and executing molecular prediction (log affinity and logP) scores. Checkpoint 2 step 1 and step 2 build on the checkpoint 1 protein sequences using composition features and BLOSUM62-inspired embeddings. Checkpoint 2 step 3 enhances the feature set further via adding Smith–Waterman alignment for sequence representation. Checkpoint 2 step 4 trains a classification model for predicting kinase family using the integrated dataset. Checkpoint 3 constructs a combined dataset by fusing EGFR patient mutation profiles (Exon 19 del, L858R, L861Q, G719X and Exon 20 mutations) with the checkpoint 1 ANN-based ligand binding estimations and checkpoint 2 EGFR family classification outputs. The LightGBM proxy model is used with the fusion dataset to compute a personalized drug response for each patient and ligand pair, putting together molecular and biological signals into patient level therapeutic insights.

3 Methods and Implementation

3.1 Checkpoint -1

Checkpoint 1 creates the baseline pipeline to predict protein ligand binding affinity. It starts by loading and filtering the dataset to include only kinase proteins. Then, protein and ligand features are generated separately. Finally, these features are combined into an ANN machine learning model and evaluated EGFR binding analysis.

3.1.1 Step-1 BindingDB Kinase Data Processing

In order to build a biologically relevant and chemically standardized kinase dataset, our team used the BindingDB data base for the downstream machine learning pipeline and transfer learning applications. The BindingDB database is a public repository that catalogs experimentally measured protein ligand binding affinities, thus providing an applicable source for our kinase driven drug discovery research. Since our pipeline is EGFR specific (UniProt ID:P00533), the entries were conserved for transfer learning evaluation.

The February 2026 BindingDB catalog was downloaded directly from their website and parsed into 50,000 row chunks by chunk-based loading to compute everything within memory limits (McKinney, 2010). In order to implement required fields such as target name, ligand SMILES, affinity metrics, and amino acid sequences, dynamic column mapping was applied to narrow schema related processing failures that may have been caused by the databases inconsistent naming conventions.

The data was cleaned by removing rows with missing chemical or biological information. Since our pipeline sequence and structure are based upon downstream modeling entries, missing target names, protein sequences, and ligand SMILES strings were kept out. Authorized data retention in BindingDB provides a wide range of assay formats, mostly focusing Ki, Kd and IC50 to conserve pharmacological importance. affinity values for Ki, Kd, IC50 were normalized by first analyzing numerical values with uneven formatting symbols (i.e. ~>, <), and later choosing the most solid available quantitative metric per entry.

In order to follow standard practice in cheminformatics and QSAR modeling, the affinity values were calculated as $-\log_{10}(\text{affinity} \times 10^{-9})$ and transformed from nanomolar concentrations to a merged negative logarithm scale pKd, pKi, pIC50. This way, standardized heterogeneous measurements are modified into an analyzable biochemical potency scale meaning higher values advise robust binding affinity (Tropsha, 2010).

Next, biological filtering was applied to obtain kinase related proteins solely, by associating target names containing "kinase" in their identifiers, while keeping EGFR rows explicitly through Uniprot based labeling, even in the presence of annotation irregularities. The double filtering approach concentrated broad kinase pathway connections while preserving a biologically crucial EGFR subset for transfer learning experiments in the following steps. Each entry was labeled with an is_egfr Boolean flag to simplify future training and test separation.

Step 1 - The output dataset contained nearly 700,000 rows of cleaned kinase specific information, including four crucial columns; protein sequence, ligand SMILES, log transformed affinity, and EGFR classification. Sanity checks, confirmed that no key columns were missing values, all affinity values were positive before logarithmic conversion, and EGFR rows were correctly labeled. This step established a reliable biological and chemical baseline dataset for following predictive modeling by stabilizing computational efficiency, biochemical standardization and target focused significance.

3.1.2 Step-2 Needleman-Wunsch Sequence Alignment of Kinase Family

Once we had a cleaned kinase dataset built from Step 1, we were able to generate sequence originated biological features to measure structural and functional similarity among kinase proteins. While Step 1 constructed our biological binding log quantification, this second stage utilized an original protein sequence itself as a supplementary source for predicting facts since the amino acid sequences guide protein folding, catalytic domains and ligand binding pocket shape. Since even the slightest mutations in kinase domains can alter ATP binding pockets or inhibitor responses, sequence comparisons will enable us to understand how chemically similar ligands may behave differently across kinase families (Manning et al., 2002).

While keeping the integrity of all kinase rows in Step 1’s dataset output, we performed a deeper cleanup of each protein sequence to achieve valid alignment scores by removing missing sequences, characters in uppercase and non alphabetic symbols. Generic labels, such as "protein", "receptor", "human" and "kinase", were removed by using regex standardization to allow only biologically informative family descriptors which could be used for grouping kinase categories for the reference panel creation.

In order to prevent EGFR biased family modeling, P00533 sequences were excluded from the reference family set. This provides greater confidence that EGFR transfer learning in the following steps stay independent and is not disproportionately effected. Further filtration includes determination of biologically significant groups with at least five members

to provide lower statistical noise from outliers and weakly portrayed categories. The longest single sequences were then selected as a primary reference for each remaining family. Selection of the longest sequence program expands the capacity of maintaining most complete catalytic and regulatory domain data, leading it to be a rigorous global reference for family wide comparisons.

Needleman Wunsch’s global sequence alignment algorithm examines protein sequences from beginning to end via Biopython’s PairwiseAligner (Needleman and Wunsch, 1970 , Cock et al., 2009). Categorization of kinase domains relies on broader domain structure, thus the global approach was a more effective process for our goal.

Alignment scoring parameters were directly tuned as follows:

- `match_score` = +1.0
- `mismatch_score` = 0.0
- `gap_opening_penalty` = -1.0
- `gap_extension_penalty` = -0.5

The parameters scored exact matches of the amino acids, limited extreme penalties for conservative replacements, and prevented redundant insertions or deletions, while permitting only biologically possible domain length combinations. The higher penalty for gap opening penalty compared to `gap extension` as -0.5 indicates an elevated biological cost for introducing new insertion or deletion regions versus the extending insertion and deletion regions.

The alignment was executed against all family reference sequences to create multiple feature sets. These include the raw Needleman Wunsch score, a `standardized Needleman Wunsch score` (i.e. calculated by dividing the raw score by the longest sequence length), prefix similarity as direct positional identity over the shared sequence length, and the mean family wide similarity metrics. The most immediate kinase family predicted the highest normalized alignment for our assessment.

It was noted that solo raw scores can inflate the similarity; therefore normalization is crucial for longer sequences (i.e sequences above 450kda). Dividing by the sequence length allows a scale-aware comparison across diverse kinase families. This step created machine learning appropriate numerical variables from sequence homology while conserving biological transparency. Through these processes we were able to extend the Step 1 dataset by adding `closest_kinase_family`, `Needleman_Wunsch_score`, `normalized_nw_score`, and `sequence_similarity`.

Due to the `sanity checks` we performed, we were able to verify that only biologically valid sequences were aligned, and normalization values kept lengths properly adjusted. Family groups ensured minimum size thresholds, while also allowing for changes to the protein evolutionary sequences. This allowed significant increases in the dataset and enabled more accurate modeling of ligand binding behavior.

3.1.3 Step-3 Building ML Ready Dataset Combining Protein + Ligand Features

Molecular properties such as molecular weight, LogP, hydrogen bond donors, and hydrogen bond acceptors represent measurable chemical characteristics of a compound. These attributes dictate how ligands behave chemically, biologically, and pharmacologically during drug and target connections. They are commonly used because molecular composition directly influences binding, absorption, and therapeutic efficacy. In Step 3, ligand SMILES strings were used as our starting chemical input, providing a compositional description of each ligand in text format. Due to raw SMILES containing formatting artifacts that normally disrupt computational analysis, a cleaning step is executed to remove unnecessary text and standardized chemical notation, and allowing higher parsing accuracy before molecular calculations begin. Right after the cleaning step, RDKit was used to convert each SMILES string into a molecular object, so the `SMILES strings`’ text was converted to a computational chemical structure. Invalid molecules were removed to maintain dataset quality. Reasonable molecular structures were crucial for creating fingerprints and descriptor extractions.

Molecular weight is extracted to calculate compound size and mass, because it influences diffusion, transport and biological accessibility. Large molecules tend to have difficulty passing membranes efficiently whereas smaller ligands have higher permeability. LogP was calculated to define `lipophilicity`, which indicates the balance between hydrophobic and hydrophilic behavior. This feature highly correlates to membrane permeability and solubility since stabilized LogP values mostly support favorable `pharmacokinetics`.

Hydrogen bond donors were measured to predict bonding capacity, because hydrogen binding groups supply to molecular recognition and receptor interactions. Additionally, hydrogen bond acceptors were also calculated for predicting complementary binding interactions, all together these structures influence ligand-protein specificity and binding affinity.

In order to convert molecular subclasses into binary numerical patterns, a system called Morgan fingerprints was created to encode systemic architecture and fingerprints. These fingerprints design each bit, showing local atomic neighborhoods

while maintaining their chemical structure and converting it into machine readable structures. These connectivity fingerprints were further noted as highly effective for molecular similarity analysis (Rogers and Hahn, 2010). The 128 Morgan fingerprint bits format balanced chemical detail so the pipeline leaves a low footprint in the RAM usage. Therefore, our design enabled large scale processing while lowering computational cost. Chunk processing continued as in Step 1 to prevent memory overload, and executing rows in batches improved scalability and stability. Each chunk was saved incrementally to maintain the progression. The Step 2 input had sequence alignment and kinase family similarity features and in Step 3 output `checkpoint_1_step_3_spam_feature_table_data.csv` added a ligand molecular descriptors and Morgan finger prints features.

In Step 1 through Step 3, the transfer learning pattern created a unified dataset merging biological sequence similarity with chemical structure information. Protein relationships and ligand molecular properties are combined in the feature table for predictive modeling of kinase ligand binding.

3.1.4 Step 4 Kinase ANN Regression Binding Affinity Model

This step of the code trains an ANN regression model called KinaseEgfrweightedANN to predict `log_affinity` for all kinase data. `X` is made by removing `target_name`, `uniprot_id`, `log_affinity`, `egfr`, and `is_egfr` from the feature table. `log_affinity` is used as a target for `y` value. The model is learning how the numerical ligand and protein features in `X_train` relate to the binding affinity values in `y_train`. The function `is_egfr(df)` creates the `is_egfr` label by searching rows to see if they contain any EGFR information through P0053, EGFR, ERB1. The code uses all kinase rows in `all_kinase_df`, then separates them into `egfr_df`, and `non_egfr_df` to track and rebalance EGFR data. The data was split into `full_train_df`, `validation_df`, and `test_df` using `train_test_split`. `make_bins()` groups `log_affinity` values into low, medium, and high binding affinity ranges so training, validation and test datasets keeps a similar distribution of high affinity values (Ozturk et al., 2018). The EGFR data was imbalanced compared to rest of the kinase data, and our downstream model, `model3`, contains EGFR specific patient mutations. Thus, EGFR is over-sampled only inside the training set using `Egfr_oversample_multiplier = 2`. It copies EGFR rows in `train_base_df` and adds them back into `train_df`. The validation and test set are not oversampled, so they can represent the real model performance. Oversampling EGFR allows the model to better learn the EGFR patterns. `Sample_weights_train = make_sample_weights(train_df)` gives EGFR rows a weight of `Egfr_class_weight = 2.5`. Non EGFR rows are weighted as 1 using `Non_egfr_class_weight = 1`. Adding weights allows our model to count EGFR mistakes more than non EGFR mistakes during training. Oversampling provides the model with more EGFR examples to learn from, while the weights make EGFR errors more important in the loss calculation. `Train_medians` is used to handle missing values safely. Using training medians for validation and test sets helps prevent the data leakage. `StandardScaler()` stabilized `X` by scaling features so they have similar ranges. `Y_train`, `y_val`, and `y_test` are also stabilized by converting `log_affinity` into standardized values using `y_mean` and `y_std`. The ANN has three layers, `dense1`, `dense2`, and `dense3`, and their sizes are controlled by three hidden layers. Each hidden layer has a RELU activation which helps the model learn non linear patterns. `L2_lambda` stabilized the training by penalizing very large weights. `SGradientD_Optimizer` updates the weights using gradient descent with momentum (He et al., 2017). `Learning_rate_pat`, and `learning_rate_decay` is used to reduce learning when validation RMSE stops improving. `R2_score`, `MSE_Loss`, and `RMSE_Loss` were used to monitor accuracy across training, validation and test datasets to calculate predictive fit and binding error. The model scored **Train** $R^2 = 0.7801$, **Validation** $R^2 = 0.6624$, and **Test** $R^2 = 0.6043$, encapsulating approximately 60% of EGFR binding variability with strong generalization and small performance gaps among unseen data. **Train** RMSE = 0.74, **Validation** RMSE = 0.91, and **Test** RMSE = 0.93. These results suggest stable convergence and biologically meaningful ligand ranking (Figure A1).

3.2 Checkpoint 2

Checkpoint 2 steps 1 and 2 transform raw kinase protein sequences into structured numerical representations through two complementary approaches; interpretable composition based features, and position aware deep embeddings. Step 3 adds local alignment features to the feature set, and then step 4 creates a kinase family classification model via XGBoost using the enhanced feature collection.

3.2.1 Step 1: Sequence Feature Extraction

Step 1 extracts 28 handcrafted features capturing what amino acids are present and their collective chemical character (Chou, 2001). The 20 amino acid composition features provide a normalized frequency vector that serves as a molecular fingerprint for distinguishing between kinase families (Hanks and Hunter, 1995). The eight physicochemical properties consist of five individual properties, hydrophobicity (Kyte-Doolittle scale (Kyte and Doolittle, 1982)), molecular weight, polarity, net charge, and volume plus three derived ratios: hydrophobic, polar, and charged residue ratios. These encode

chemical characteristics directly relevant to drug binding. Hydrophobicity governs ligand occupancy in ATP-binding pockets through van der Waals interactions. Charge distribution determines electrostatic complementarity with ATP and other inhibitor molecules. Polarity dictates hydrogen-bonding capacity and solubility. Volume captures steric constraints that can permit or block ligand access (Grantham, 1974). Together, these interpretable features link sequence variation to functional changes in EGFR mutations, where even single residue alterations can reshape conformation and drug affinity.

3.2.2 Step 2: Deep Sequence Embeddings

Step 2 generates 64-dimensional embeddings that capture how residues are arranged, complementing Step 1’s "what is present" with "where it appears." The 8-dimensional amino acid vectors are manually specified and informed by BLO-SUM62 substitution patterns (Henikoff and Henikoff, 1992), encoding evolutionary relationships so that biochemically similar residues receive similar representations. Although this hand-crafted reduction from a 20×20 substitution matrix to 8 dimensions entails information loss, it yields a compact, interpretable embedding that captures the most salient biochemical groupings without requiring external pretrained models. Position-weighted aggregation emphasizes the N-terminal and C-terminal regions, areas rich in localization signals, auto inhibitory domains, and drug-binding sites (Hanks and Hunter, 1995). Multi-statistic pooling (mean, standard deviation, maximum, minimum) captures both central tendency and variability across the sequence, identifying regions of chemical heterogeneity that may correspond to binding pockets or domain transitions (Kawashima et al., 2008). The concatenation of all embedding components (mean, std, max, min, position-weighted, N-terminal, C-terminal and each 8 dimensions) plus a normalized sequence length yields a 65-dimensional vector, which is truncated to 64 dimensions to produce a fixed-size input for downstream models. Although the truncated length feature sacrifices one dimension of information, sequence length is partially preserved through the position-weighted and terminal embeddings and is already captured separately in Step 1, minimizing the impact on downstream performance. This enables the model to recognize functional analogies between kinases with different compositions but similar spatial and chemical profiles.

Steps 1 and 2 are complementary in that the composition features offer interpretable and direct physicochemical insight to understand the effects of mutation on drug binding, while the embeddings capture spatial organization and evolutionary patterns that composition alone cannot represent. Their combination provides a robust dual representation that supports both explainable analysis and high precision classification for EGFR targeted precision medicine (Alley et al., 2019).

Checkpoint 2 step 1 - Output

The code generates a feature enriched dataset containing both amino acid composition features and physicochemical properties. Each kinase sequence in the input data set is transformed into a comprehensive feature vector containing:

1. **Amino acid composition features (20 features):** Each of the 20 standard amino acids (A, C, D, E, F, G, H, I, K, L, M, N, P, Q, R, S, T, V, W, Y) receives a feature labeled `aa_X` where X is the amino acid single letter code. These features represent the normalized frequency of each amino acid in the sequence, summing to 1.0 across all 20 features. For example, if a kinase sequence contains 8% alanine, the `aa_A` feature will be 0.08. These composition frequencies provide a complete description of the building block distribution in each kinase.
2. **Physicochemical features (8 features):**
 - `mean_hydrophobicity`: Average Kyte-Doolittle hydrophobicity score across the sequence
 - `mean_molecular_weight`: Average molecular weight of amino acids in the sequence
 - `mean_polarity`: Average polarity value (0 or 1 for non-polar/polar residues)
 - `net_charge`: Sum of individual amino acid charges (-1, 0, or +1)
 - `mean_volume`: Average volume of amino acid side chains
 - `hydrophobic_ratio`: Fraction of residues with positive hydrophobicity
 - `polar_ratio`: Fraction of polar residues in the sequence
 - `charged_ratio`: Fraction of residues with non-zero charge
3. **Sequence length feature (1 feature):**
 - `seq_length`: Total number of amino acids in the cleaned sequence

The output data set saves this enriched information to `checkpoint_2_step_1_sequence_feature_data.csv`, preserving all original columns from the input while adding 29 new feature columns (20 amino acid compositions + 8 physicochemical properties + 1 sequence length). This dataset serves as the foundation for downstream machine learning tasks, including kinase family classification and binding affinity prediction.

Checkpoint 2 step 2 - Output

Step 2 generates a comprehensive set of deep embeddings features that complement the composition features from Step 1. The output includes:

1. **New embedding features (64 features):** Each embedding vector is labeled `emb_0` through `emb_63`, creating a fixed-length 64-dimensional representation for each kinase sequence. These embeddings combine multiple statistical views of the sequence:
 - Mean embedding (8 dimensions): Average amino acid embedding across the sequence
 - Standard deviation embedding (8 dimensions): Variability of embeddings across positions
 - Maximum embedding (8 dimensions): Maximum value for each embedding dimension
 - Minimum embedding (8 dimensions): Minimum value for each embedding dimension
 - Position-weighted embedding (8 dimensions): Weighted average emphasizing the terminal
 - N-terminal embedding (8 dimensions): Average of first 10 amino acids
 - C-terminal embedding (8 dimensions): Average of last 10 amino acids
 - Length feature (1 dimension): Normalized sequence length
2. **Dataset integration:** The output file `checkpoint_2_step_2_deepseq_embedding.csv` contains all original columns from Step 1 output plus the 64 new embedding columns. This creates a unified dataset with:
 - 20 amino acid composition features
 - 8 physicochemical property features
 - 1 sequence length feature
 - 64 deep embedding features

The combined feature set provides both interpretable domain-relevant features (composition and physicochemical) and learned representations (embeddings) that capture complex patterns beyond the manual feature design. This dual approach aligns with the goals of precision medicine by providing features that clinicians can interpret (composition, hydrophobicity, charge) while also leveraging the predictive power of deep representations to improve binding accuracy for EGFR mutations.

3.2.3 Checkpoint 2 Step 3 - Local Alignment Features and Hierarchical Clustering Visualizations

Step 3 integrates local alignment scores into the feature set to improve the characterization of kinase proteins and better capture the relationships between kinase families.

Method Review

Local alignment features were incorporated into the feature set to further enhance protein representation. Unlike the Needleman-Wunsh global alignment method, which compares two entire sequences from beginning to end, local alignment focuses only on the most similar subsections of proteins. This is an important distinction because proteins may differ substantially overall, but can still share short functional regions that control their biological activity (Kim et al., 2020).

For kinase proteins, these conserved local regions are especially important because they often correspond to catalytic domains, ATP-binding regions, and ligand-binding sites, as well as other functional motifs involved in signaling pathways. Mutations can also affect only small regions of the protein rather than the whole sequence, making local alignment particularly useful for identifying the biologically meaningful similarities that are overlooked using only global alignment features.

In this project, local alignment scores were computed using the Smith-Waterman algorithm. The Smith-Waterman algorithm computes the local alignment scores where having a higher local alignment score between proteins would suggest a greater likelihood of having similar ligand interaction behavior (Figure B1). To see whether the integrated features capture meaningful biological structure, unsupervised learning and visualization techniques were applied. Hierarchical clustering was used to visually evaluate the relationships between proteins and their kinase families. The clustering process begins by treating each kinase family as an independent group, where families with the most similar features are then merged step-by-step, forming larger clusters. The resulting relationships are visualized using a dendrogram, a tree-like diagram showing how closely related kinase families are to one another. Branches that merge at shorter distances indicate a stronger similarity, while branches that merge farther apart indicate greater divergence. Hierarchical clustering was used

to generate a family-level dendrogram(Figure B2) based on the feature similarity. This dendrogram helped to visualize how closely related some kinase families might be based on our feature set, informing for the step 4 classification.

Implementation

The implementation of the Smith-Waterman algorithm and hierarchical clustering followed a structured computational pipeline that was designed to efficiently generate local alignment features and integrate them with previously engineered protein representations. The process began by loading the Checkpoint 2 Step 2 dataset containing protein sequences, sequence embeddings, amino acid composition features, physicochemical properties, and global alignment metrics. The protein sequences were first cleaned and standardized to ensure consistency throughout the analysis. Invalid characters and non-standard amino acid symbols were removed, leaving only valid amino acid residues. This cleaning step, while redundant from previous steps, was important because sequence alignment algorithms require properly formatted biological sequences to generate accurate scores, so these additional safety checks were implemented. Representative reference sequences were then constructed for each kinase family. Duplicate sequences were removed and a small number of representative proteins from each family were selected to create a biologically diverse reference set. These reference proteins serve as a reference point for local sequence alignment. For each protein sequence in the dataset, the local alignment scoring was performed against the reference set using pairwise local alignment scores from the Smith-Waterman algorithm in Bio-Python with the following scoring parameters:

- match = +2 rewards matches, giving +2 to score
- mismatch = -1 penalizes mismatch, giving -1 to score
- open gap = -1 penalizes opening a new gap (gap it an insertion/deletion), giving -1 to score
- extend gap = -1 penalizes adding to an existing gap, giving -1 to score

This algorithm generates a matrix of similarity scores between each sequence and the reference set. From these alignment results, multiple summary features could be extracted to help capture different aspects of local similarity:

```
local_alignment_best_score:
    Highest alignment score that is observed across all reference sequences, indicating strongest
    local similarity

local_alignment_mean_score:
    Mean alignment score across all comparisons, capturing overall similarity trends.

local_alignment_best_normalized:
    Best alignment score normalized by sequence length

local_alignment_mean_normalized:
    Normalized average score

local_alignment_score_std:
    Standard deviation of alignment scores showing the variability in similarity across kinase
    families.

local_alignment_high_count:
    Number of alignments exceeding a predefined similarity threshold, indicating frequency of
    meaningful matches

local_alignment_best_match_family:
    Kinase family corresponding to the highest alignment score, functioning as a pre-classifier
    for family membership
```

These features provide an improved representation of how each protein relates to known kinase families at the local sequence level.

After computing the local alignment features, they were integrated with previously generated feature sets, including amino acid composition and physicochemical properties (Step 1), deep sequence embeddings (Step 2) and global alignment features (Checkpoint 1). This combined feature space can allow the model to capture multiple points of biological information, with the integration of local alignment scores being particularly valuable for identifying subtle functional similarities that may not be captured by global alignment.

Important sanity checks were performed throughout the implementation to ensure data quality and computational correctness. All sequences were verified to successfully produce alignment scores, and no missing or infinite values remained within the final feature matrix. Alignment scores were normalized by sequence length to allow fair comparison across proteins of different sizes. Reference sequence mappings were validated to ensure correct kinase family assignments, and feature dimensions were checked to confirm consistency across all samples.

To generate the family-level dendrogram (Figure B2), proteins were grouped by their kinase family and median feature values were computed to represent each family. The features were standardized before hierarchical clustering was applied using Ward linkage, which groups clusters by minimizing the variance within them. The resulting clustering and visualization output demonstrated that proteins from the same kinase families were grouped generally together, but there was potential for overlap between families due to similar biological functions.

3.2.4 Checkpoint 2 Step 4

This step implements a supervised machine learning model to tackle protein kinase family membership using the integrated feature set from previous steps.

XGBoost Classifier

Method Review

The XGBoost (eXtreme Gradient Boosting) Classifier is a supervised machine learning algorithm based on gradient boosting of decision trees, an ensemble learning technique that works by building a sequence of decision trees in a stepwise manner. Unlike a single decision tree, which may overfit or fail to capture complex relationships. Gradient boosting combines many shallow trees, often referred to as "weak learners" to produce a more predictive model. Each new tree attempts to correct the prediction errors made by the previous trees, allowing the model to gradually improve performance over multiple iterations. The XGBoost design is useful for achieving higher predictive performance while also maintaining computational efficiency with built-in methods for handling overfitting (T. Chen and Guestrin, 2016). The key idea behind gradient boosting is iterative error correction. The model is built sequentially, where each new tree is trained to correct the mistakes made by the previous ensemble of trees. The model begins with an initial prediction and iteratively improves the performance by minimizing classification error through gradient optimization. This process allows XGBoost to capture the complexity or nonlinear relationships between the features. Given how diverse the dataset is, including sequence-based, structural, and physicochemical features, this ability is essential for accurately modeling patterns. In this project, XGBoost was chosen to classify proteins into kinase families, a task characterized by complex feature relationships and an imbalanced feature set. XGBoost improved other decision tree methods by introducing several optimizations, such as regularization, learning rate, parallelization, and tree pruning. These enhancements make XGBoost efficient, robust, and a perfect match for high-dimensional datasets like the one used in this project.

Implementation

The classification pipeline was implemented using the integrated feature set generated in the prior steps. Data were checked for consistent formatting, removing missing rows of kinase family labels or UniProt identifiers to ensure valid classification targets and proper grouping of proteins. Additional pre-processing steps standardized the family labels and UniProt IDs by removing blank entries and inconsistent formatting. To help reduce the instability caused by extremely small classes, kinase families with insufficient protein representation were filtered from the dataset. Families were required to have a minimum number of unique proteins before inclusion in the classification analysis.

To reduce the redundancy within the dataset, features were aggregated at the protein-level using median feature values for each UniProt identifier. This aggregation prevented proteins with repeated measurements to disproportionately influence model training and ensured that each protein contributed more equally during the classification. Protein-level holdout splitting was then applied using UniProt IDs as grouping variables. Entire proteins were assigned exclusively to either training or testing datasets, ensuring zero overlap between the two groups. This represented one of the most important sanity checks in the pipeline because it prevented data leakage between training and evaluation datasets. After splitting, overlap between training and testing proteins was explicitly verified to confirm complete separation. Kinase family labels were encoded numerically using label encoding methods to allow compatibility with the XGBoost classifier. To address the class imbalance between kinase families, balanced sample weights were computed and applied during training so that underrepresented classes contributed appropriately during optimization.

The XGBoost classifier was trained using gradient boosted decision trees with histogram-based tree construction, feature sub-sampling, learning-rate control, regularization penalties, and depth constraints to improve computational efficiency and reduce overfitting. Initial model training was followed by feature selection using model-derived importance scores. Features with importance scores above the mean threshold were retained, and the classifier was retrained using the reduced feature set to improve efficiency and reduce noise. Important sanity checks performed were verification of the feature matrices containing only numeric values with no missing or infinite entries. Label consistency between training and testing datasets were validation, and majority-class baseline accuracy was computed to ensure the classifier performed

meaningfully above random or trivial predictions. Model convergence and training accuracy were monitored for overfitting detection, which was the main concern throughout this implementation. Predictions and confidence probabilities were generated for the held-out protein test set. Classification metrics, confusion matrices, feature important ranking, and prediction confidence distributions were generated for confirmation, downstream analysis, and overall interpretation.

3.3 Checkpoint 3

Checkpoint 3 connects patient EGFR mutation profiles with ligand binding predictions and EGFR kinase family classification outputs to create personalized fusion dataset for treatment prediction. Checkpoint 3 model 3 (LightGBM Model) learns from a fusion of patient mutation and binding data to predict drug response for each patient ligand pair, enabling personalized treatment insights.

3.3.1 Checkpoint 3 Step 1 : Patient Mutation Dataset Construction

By structuring the data at the patient level and encoding the presence of mutations as features, this step establishes a bridge between traditional molecular modeling and personalized medicine. Rather than treating EGFR as a uniform target, our dataset reflects how its behavior varies across individuals, enabling downstream models to incorporate patient-specific biological context in Lung Adenocarcinoma.

To construct the final modeling dataset, we implemented a multi-step data integration pipeline combining data from ChEMBL, The Cancer Genome Atlas (TCGA), and Clinical Proteomic Tumor Analysis Consortium (CPTAC). Mutation data was grouped by patient ID and filtered to include only EGFR-related mutations. Across datasets, we unified sample identifiers, standardized gene naming conventions, and preserved cohort-specific batch labels to maintain data provenance. For patients with multiple mutations, mutation profiles were categorized based on clinically relevant EGFR hotspots, including exon 19 deletions, L858R, L861Q, and G719X. This process resulted in a curated dataset of approximately 70 unique patients, each represented by structured mutation features suitable for downstream modeling. Additional biological features were integrated at the patient level, including EGFR expression, protein abundance, phosphorylation site measurements, and a derived EGFR activity score based on functionally relevant phosphosites.

patient_id	has_exon19del	has_L858R	has_L861Q	has_G719X	is_egfr_hotspot
TCGA-05-4382-01	0	0	0	0	0
TCGA-05-4402-01	1	0	0	0	1
TCGA-05-4410-01	0	0	0	0	0
TCGA-05-5423-01	0	0	1	0	1
TCGA-17-Z026-01	0	0	0	0	0

To extend this dataset toward predictive modeling, ligand-level chemical information was later incorporated using SMILES representations obtained from ChEMBL. Each SMILES string was converted into quantitative chemical descriptors providing a numerical representation of the drug structure (molecular weight, logP, and hydrogen properties). These ligand features were then merged with the patient mutation dataset to form a fused patient–ligand modeling table; thus, the modeling pipeline was able to capture both biological "mutation effects" and chemical "drug properties" variations. Because EGFR mutations can alter binding site structure, these combined features allow the model to learn how mutation-specific changes influence drug affinity. Rather than memorizing individual drug responses, the model learns generalizable patterns between chemical descriptors and mutation-aware protein features. This enables the prediction of binding affinity across multiple drugs and patient profiles, forming the foundation for the modeling approach implemented in Step 3.

3.3.2 Checkpoint 3 Step 2 : Fusion Model

The main focus in this step is to fuse biological, chemical and bioinformatics data into one personalized data matrix. First checkpoint 3 step 1 data was loaded for 71 patient EGFR mutation profiles from `checkpoint_3_patient_mutation_data.csv`. In this dataset each patient had multiple EGFR mutation patterns, mutation burdens, hotspot labels, and binary mutation indicators. Respectively, mutation burden is the total number of mutations in a patient, while hotspot labels represent key EGFR mutation sites strongly linked to cancer activity and drug response. Next, checkpoint 1 data loaded all kinase ligand rows from `checkpoint_1_step_3_No_smiles_spam_feature_table_data.csv`. Indeed, these ligands are considered small molecules, and all included chemical descriptors such as `molec_wt`, `logP`, `hydrogen_donor`, `hydrogen_acceptor` and protein sequence similarity variables such as `normalized_nw_score` and `sequence_similarity`. (He et al., 2017)

To predict ligand binding affinity for all kinase compounds, the function `model1_score` reused the checkpoint1 ANN model with `model1_xscaler_path` and `model1_yscaler_path`, generating `model1_binding_pred` values. Thus, the model always predicts proxy ligand binding strength, which later becomes combined with patient data during the cartesian join. (Jiang et al., 2023) The ANN model was retrained using all kinase data with EGFR weighting and EGFR oversampling.

The generated predictions represent proxy binding affinity scores instead of experimental biological labels. Rows were labeled using `is_proxy_affinity_score = 1` and `is_simulated_pair = 1`. This helped reduce circular learning and target leakage because patient mutation data was added only after ANN prediction generation.

To prevent feature imbalance, `x_scaler` standardizes the input features to the same scale used during ANN training. Additionally, `y_scaler`, of ANN model1, is used to convert predictions back to the true biological binding scale so they can be explained accurately.

Checkpoint 2 transfer learning predictions were cleaned using `prepare_cp2_predictions` keeping kinase family outputs such as `predicted_family`, `prediction_confidence`, `true_family`, and `closest_kinase_family`.

Checkpoint 1 step 3 kinase data and Checkpoint 2 family predictions were merged using shared identifiers such as `uniprot_id`, `target_name` or `gene`.

The most crucial phase was `cartesian_join(cp3, cp1_2_fused)`. In this part of the code, every possible patient x ligand combination matrix was created. Therefore, each of 71 patients were matched with all kinase ligands creating nearly a largely simulated interaction dataset. This is a proxy simulation dataset represents every possible patient drug interaction biologically. Ligand molecular properties were encoded using SMILES derived descriptors and Morgan fingerprints to capture how molecular features may behave across different EGFR mutation backgrounds while reflecting the chemical properties of each compound. From a bioinformatics point of view, this transformed separate datasets into treatment response matrices.

Lastly, `last_feature_cleanup` created interaction variables such as `proxy_model1_x_hotspot` and `proxy_model1_x_mut_count`. All these features related to predicted drug binding with mutation severity. In the dataset each final row indicates one patient ligand pair with mutation, ligand, sequence, and predicted binding features. This fusion dataset became the base for personalized proxy drug response prediction in checkpoint 3 proxy LightGBM model (`model3`).

3.3.3 Checkpoint 3 Step 3 : LightGBM Proxy Model for Binding Predictions

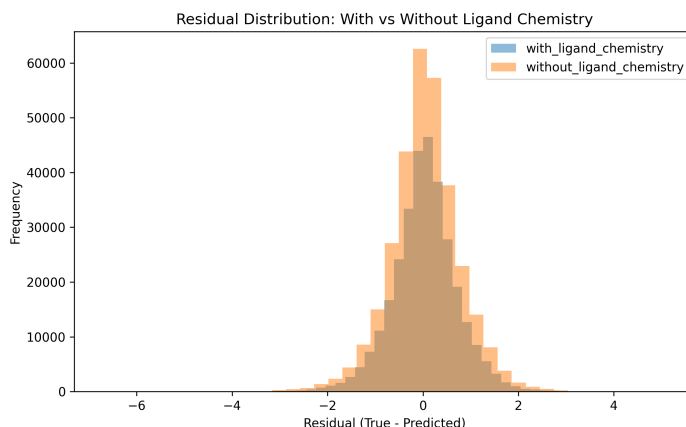


Figure 1: Residual distribution comparison between the LightGBM proxy models with and without ligand chemistry features. The ligand-enhanced model produced a narrower distribution centered closer to zero, indicating improved predictive stability and reduced error.

Based on the patient-ligand fused dataset, the goal for this step was to integrate mutation-derived biological features with ligand chemical descriptors to model both the biological variation in EGFR and the chemical variation in the target inhibitors. With a fixed y variable of the binding affinities, the model can represent the strength of interaction between the given drug and the EGFR protein under the specific mutation profile. A LightGBM regression model was implemented to predict the continuous binding affinity values in which the interactions between indicators and ligand descriptors can capture these complex, non linear relationships(Ke et al., 2017). The model was configured with 500 estimators and a learning rate of 0.05, balancing the model complexity with generalization performance while reducing possible overfitting to noise in the dataset.

The fusion modeling stage combined 71 patient mutation profiles with 697,624 kinase-related ligands, generating nearly 48,833,680 patient-ligand interaction rows. Chemical descriptors derived from SMILES representations, including molecular weight, logP, hydrogen bonding properties, and molecular fingerprints, were integrated with mutation-specific and kinase-family features. This allowed the dataset to represent both biological mutation effects and ligand structural behavior simultaneously. The final LightGBM regression model showed that the incorporation of ligand chemistry improved

predictive performance compared to the mutation-only model. The baseline model achieved a test R^2 of 0.814 with an RMSE of 0.401 and MAE of 0.2132, while the ligand-enhanced model improved to a test R^2 of 0.861 with reduced RMSE (0.684) and MAE (0.513). Residual distribution analysis further showed a narrower error distribution centered closer to zero, indicating more stable predictions across unseen patient profiles.

A key outcome of this model was the identification of high predicted proxy affinity predictions drug-patient combinations, representing inhibitors that may exhibit stronger interactions under specific mutation conditions. In practice, these predictions could guide experimental validation efforts such as cell-based assays or inhibitor screening studies. In some cases, multiple high-performing inhibitors may eventually be explored as potential *drug cocktail* combinations to address resistance pathways and improve long-term therapeutic effectiveness.

4 Model Evaluations and Results

4.1 Checkpoint 1 Step 4

See Appendix A for ANN model graphs and explanations.

4.2 Checkpoint 2 Step 4

Kinase Family Classification

The performance of the classification model was evaluated using several different metrics. Accuracy was used to measure overall classification performance, while F1 scores (macro and weighted) were calculated to account for class imbalance and provide a more balanced evaluation across kinase families (Figure B3). The confusion matrix (Figure B4) provides a detailed view of the correct and incorrect predictions, with a strong diagonal indicating an accurate classification. Off diagonal values highlight the specific families that are frequently misclassified, offering insight into areas where biological function similarities may be leading to overlap. Feature importance scores were also analyzed to determine which features contributed the most to the model predictions (Figure B5). These results help validate that biologically meaningful features play a significant role in classification. An additional evaluation included a confidence score analysis to assess the accuracy of the prediction. High confidence predictions indicate strong model certainty, while lower confidence prediction may reflect ambiguous cases or overlapping biological features (Figure B6).

The XGBoost model showed signs of severe to moderate overfitting throughout the development process, ending with a training accuracy of 90% and a testing accuracy of 72%. This overfitting may be due to the high-dimensional feature set relative to the number of unique proteins available after protein-level aggregation. Although the original dataset contained many rows, repeated measurements were grouped by UniProt ID, reducing the effective number of independent training examples. The model also used a strict protein-level holdout split, which meant that test proteins were completely unseen during training. This made the evaluation more biologically realistic, but also more difficult. The combination of having high-dimensional features with a limited number of unique proteins likely allows the model to learn training-specific patterns rather than fully generalizable kinase-family relationships. Additionally, similarity-based alignment features can strongly separate training examples, while transferring less effectively to held-out proteins from closely related or overlapping kinase families. Future directions for this model would be to change protein handling or extend the protein dataset to see if the overfitting gap between training and testing accuracies could be bridged.

XGBoost Kinase Family Classifier Results

Train Accuracy	Test Accuracy	F1 Macro Score	F1 Weighted	Features
90.48%	72.33%	0.71	0.73	105

Table 1: Output metrics from XGBoost Classifier for Kinase Family Membership

4.3 Checkpoint 3

The modeling pipeline demonstrates how biological, chemical, and machine learning workflows can be integrated into a personalized EGFR treatment prediction framework. Data cleaning and preprocessing were essential for standardizing TCGA, CPTAC, and ChEMBL datasets through unified patient identifiers, mutation annotations, and cohort labels. Clinically relevant EGFR mutations, such as exon 19 deletions and L858R were encoded into structured patient-level features alongside expression, phosphorylation, and protein abundance.

LightGBM model	features	categorical features	RMSE	MAE
without ligand chemistry	161	12	0.741421	0.550190
with ligand chemistry	166	12	0.684145	0.512998

Table 2: Output metrics from Proxy Binding Affinity LightGBM Modelings

The fusion modeling stage combined 71 patient mutation profiles with 28,544 EGFR-related ligands, generating nearly 2 million patient-ligand interaction rows. Chemical descriptors derived from SMILES representations, including molecular weight, logP, hydrogen bonding properties, and molecular fingerprints, were integrated with mutation-specific and kinase-family features. This allowed the dataset to represent both biological mutation effects and ligand structural behavior simultaneously. The final LightGBM regression model showed that incorporation of ligand chemistry improved predictive performance compared to that of the mutation-only model. Feature importance analysis identified ligand chemistry descriptors, sequence similarity metrics, and mutation-specific features as major contributors to the model. Overall, the results suggest that the combination of mutation-aware biological information with ligand structural descriptors improves the prediction of EGFR inhibitor binding behavior and provides a stronger approximation toward personalized drug-response modeling.

Proxy Model

A major limitation of this project was the lack of experimentally measured binding affinity data for a large number of patient-specific EGFR mutations. To address this, a proxy-based modeling strategy was implemented. Instead of directly predicting patient-specific drug response from raw mutation data alone, an intermediate machine learning model was first trained to estimate biologically relevant EGFR family behavior using mutation-derived and sequence-related features. The outputs from this earlier checkpoint model were then incorporated as engineered features into the final LightGBM regression framework.

Features such as predicted behavior of the kinase-family, mutation similarity metrics, and normalized sequence alignment scores served as biologically informed representations of EGFR structural variation.

5 Discussion

Project Limitations and Future Directions

A challenge in this project was the initial implementation for a cold-start modeling project, where predictions were made without prior experience in protein-specific modeling. This approach relied on similar metrics that were useful for our final product, including sequence alignments and protein relationships across kinase families. The cold-start model was insufficient to model EGFR binding behaviors, with overall model performance indicating a low prediction power and led to the following limitations: First, the difference in kinase scaffold across protein families introduced variability in binding sites that are not represented solely by sequence alignment. Additionally, steric hindrance effects that influenced how a ligand fit in a binding site "pocket" were not encoded into the sequence-based features. Lastly, our alignment metric would need to be adapted for EGFR dimerization. In the post-pivot from our initial cold-start ambitions, our next limitation encountered was in the transition from Checkpoint 2 to Checkpoint 3. Data leakage and the risk of circular model design were an important consideration throughout the pipeline construction process. During the later stages of the pipeline review, it became apparent that the study design may contain fundamental methodological limitations. Specifically, the ANN used to generate predicted binding affinities was reused to create features for the downstream proxy LightGBM model. It became clear that the ANN would have needed to be trained on non-EGFR datasets in order to not unintentionally leak training data into the proxy LightGBM model, which artificially inflates performance metrics. Additionally, the generated dataset consisted of a Cartesian expansion of 71 patients with 700,000 ligands, resulting in nearly 48 million patient-ligand combinations. However, due to RAM limitations, only 1 million fused dataset rows were used to train the LightGBM model (see Appendix D for CPU usage across all models). These combinations are not representative of experimentally validated observations, but rather of simulated pairings. Consequently, each row in the dataset should be interpreted as a synthetic data point rather than as a true biological measurement. Another limitation is that the target variable, `log_affinity`, was derived from ANN predictions rather than experimentally measured binding affinities. As a result, the proxy LightGBM model is being trained to reproduce the ANN generated output instead of learning directly from binding data. Therefore, reported model performance reflects the agreement between the upstream (checkpoint 1) ANN predictions rather than predictive accuracy for real ligand binding behaviors. This is important when interpreting the biological relevance of the model, as well as its generalization. Future work should prioritize validation against experimentally measured affinity data and ensure that there is strict separation between the upstream model training data and the downstream evaluation datasets to minimize the risk of model circularity and data leakage. Once a more stable model is produced, more advanced protein representation methods, such as ProtBERT or ProtT5 could improve feature quality by encoding more diverse

biological information. The modeling pipeline also excluded environmental variables due to a lack of reliable data sources. Future work incorporating environmental conditions and models that could improve predictive accuracy could provide a more complete representation of real-world biochemical interactions.

6 Conclusion

The final model output from the fused Checkpoint 3 LightGBM pipeline reported proxy patient ligand binding predictions `y_pred` as their order of priority. The top ranked results pinpoint several high affinity ligands along EGFR related and mutation specific patient profiles e.g., TCGA55A57B01, TCGA50659101, TCGA50659101. Ligands include **benzothiophenephenol analogs, thienopyrimidine kinase inhibitor analogs, genistein derivatives** and related to kinase associated molecules; in many cases, they are directed against biologically relevant proteins, including **EGFR, ERBB1 kinase, VEGFR, protein tyrosine kinases, and carbonic anhydrase targets**.

We show that the predicted proxy binding affinity scores with the highest predictive significance `y_pred = 11.55 to 11.71` are consistent with robust simulated ligand target affinity and possible inhibition of cancer associated signaling pathways. These results spotlight potential compounds to exert inhibition of abnormal EGFR driven signaling in lung cancer mutation settings.

The top ten ranked ligand table lists the results for the strongest binding probability for a specific patient and is displayed in Appendix C for detailed patient level analysis. These drug numbers we used are from BindingDB tsv file and each ligand is paired with their IUPAC names. Unfortunately, we can not claim that the ligand pairs with given patients could be more likely to stop the tumor growth. Since our **proxy binding affinity model only achieves 86%** accuracy, it is not ready for direct clinical use. However, it is baseline that provides the foundation for further research and refinement. We can achieve more tailored models with distinct patient profiles by adding more data, then this pipeline could be able to guide us toward more accurate, personalized predictions in the future.

References

- Abdelkader, G. A., & Kim, J.-D. (2024). Advances in protein-ligand binding affinity prediction via deep learning: A comprehensive study of datasets, data preprocessing techniques, and model architectures. *Current Drug Targets*, 25(15), 1041–1065. <https://doi.org/10.2174/0113894501330963240905083020>
- Alley, E. C., Khimulya, G., Biswas, S., AlQuraishi, M., & Church, G. M. (2019). Unified rational protein engineering with sequence-based deep representation learning. *Nature Methods*, 16, 1315–1322. <https://doi.org/10.1038/s41592-019-0598-1>
- Chen, J., Zeng, F., Forrester, S. J., Eguchi, S., Zhang, M.-Z., & Harris, R. C. (2016). Expression and function of the epidermal growth factor receptor in physiology and disease. *Physiological Reviews*, 96(3), 1025–1069. <https://doi.org/10.1152/physrev.00030.2015>
- Chen, T., & Guestrin, C. (2016). Xgboost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794. <https://doi.org/10.1145/2939672.2939785>
- Chou, K.-C. (2001). Prediction of protein cellular attributes using pseudo-amino acid composition. *Proteins: Structure, Function, and Genetics*, 43(3), 246–255. <https://doi.org/10.1002/prot.1035>
- Cock, P. J. A., Antao, T., Chang, J. T., Chapman, B. A., Cox, C. J., Dalke, A., Friedberg, I., Hamelryck, T., Kauff, F., Wilczynski, B., & de Hoon, M. J. L. (2009). Biopython: Freely available python tools for computational molecular biology and bioinformatics. *Bioinformatics*, 25(11), 1422–1423. <https://doi.org/10.1093/bioinformatics/btp163>
- Grantham, R. (1974). Amino acid difference formula to help explain protein evolution. *Science*, 185(4154), 862–864. <https://doi.org/10.1126/science.185.4154.862>
- Hanks, S. K., & Hunter, T. (1995). The eukaryotic protein kinase superfamily: Kinase catalytic domain structure and classification. *The FASEB Journal*, 9(8), 576–596. <https://doi.org/10.1096/fasebj.9.8.7768349>
- He, T., Heidemeyer, M., Ban, F., Cherkasov, A., & Ester, M. (2017). Simboost: A read-across approach for predicting drug-target binding affinities using gradient boosting machines. *Journal of Cheminformatics*, 9(1), 24. <https://doi.org/10.1186/s13321-017-0209-z>
- Henikoff, S., & Henikoff, J. G. (1992). Amino acid substitution matrices from protein blocks. *Proceedings of the National Academy of Sciences*, 89(22), 10915–10919. <https://doi.org/10.1073/pnas.89.22.10915>
- Hyman, D. M., Taylor, B. S., & Baselga, J. (2017). Implementing genome-driven oncology. *Cell*, 168(4), 584–599. <https://doi.org/10.1016/j.cell.2016.12.015>
- Jiang, M., Shao, Y., Zhang, Y., Zhou, W., & Pang, S. (2023). A deep learning method for drug-target affinity prediction based on sequence interaction information mining. *PeerJ*, 11, e16639. <https://doi.org/10.7717/peerj.16639>
- Kawashima, S., Pokarowski, P., Pokarowska, M., Kolinski, A., Katayama, T., & Kanehisa, M. (2008). Aaindex: Amino acid index database, progress report 2008. *Nucleic Acids Research*, 36(Database issue), D202–D205. <https://doi.org/10.1093/nar/gkm998>
- Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T.-Y. (2017). Lightgbm: A highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems (NIPS)*, 3146–3154. <https://doi.org/10.5555/3294771.3294806>
- Kim, J., Ji, M., & Yi, G. (2020). A review on sequence alignment algorithms for short reads based on next-generation sequencing. *IEEE Access*, 8, 189811–189822. <https://doi.org/10.1109/ACCESS.2020.3031159>
- Kobayashi, S., Boggon, T. J., Dayaram, T., Jänne, P. A., Kocher, O., Meyerson, M., Johnson, B. E., Eck, M. J., Tenen, D. G., & Halmos, B. (2005). Egfr mutation and resistance of non small-cell lung cancer to gefitinib. *New England Journal of Medicine*, 352(8), 786–792. <https://doi.org/10.1056/NEJMoa044238>
- Koulouris, A., Tsagkaris, C., Corriero, A. C., Metro, G., & Mountzios, G. (2022). Resistance to tkis in egfr-mutated non-small cell lung cancer: From mechanisms to new therapeutic strategies. *Cancers*, 14(14), 3337. <https://doi.org/10.3390/cancers14143337>
- Kyte, J., & Doolittle, R. F. (1982). A simple method for displaying the hydropathic character of a protein. *Journal of Molecular Biology*, 157(1), 105–132. [https://doi.org/10.1016/0022-2836\(82\)90515-0](https://doi.org/10.1016/0022-2836(82)90515-0)
- Liao, Y.-L., Li, Y.-C., Chen, N.-C., & Lu, Y.-C. (2018). Adaptively banded smith-waterman algorithm for long reads and its hardware accelerator. *2018 IEEE 29th International Conference on Application-specific Systems, Architectures and Processors (ASAP)*, 1–9. <https://doi.org/10.1109/ASAP.2018.8445105>
- Manning, G., Whyte, D. B., Martinez, R., Hunter, T., & Sudarsanam, S. (2002). The protein kinase complement of the human genome. *Science*, 298(5600), 1912–1934. <https://doi.org/10.1126/science.1075762>
- McKinney, W. (2010). Data structures for statistical computing in python.
- Needleman, S. B., & Wunsch, C. D. (1970). A general method applicable to the search for similarities in the amino acid sequence of two proteins. *Journal of Molecular Biology*, 48(3), 443–453. [https://doi.org/10.1016/0022-2836\(70\)90057-4](https://doi.org/10.1016/0022-2836(70)90057-4)

- Ozturk, H., Ozkirimli, E., & Ozgur, A. (2018). Deepdta: Deep drug–target binding affinity prediction. *Bioinformatics*, 34(17), i821–i829. <https://doi.org/10.1093/bioinformatics/bty593>
- Rogers, D., & Hahn, M. (2010). Extended-connectivity fingerprints. *Journal of Chemical Information and Modeling*, 50(5), 742–754. <https://doi.org/10.1021/ci100050t>
- Tropsha, E. M., Denis Fourches. (2010). Trust, but verify: On the importance of chemical structure curation in cheminformatics and qsar modeling research. *Journal of Chemical Information and Modeling*, 50(7), 1189–1204. <https://doi.org/DOI:10.1021/ci100176x>

Appendix A - (Checkpoint 1)

Checkpoint 1 Step 4 - EGFR Only ANN Results

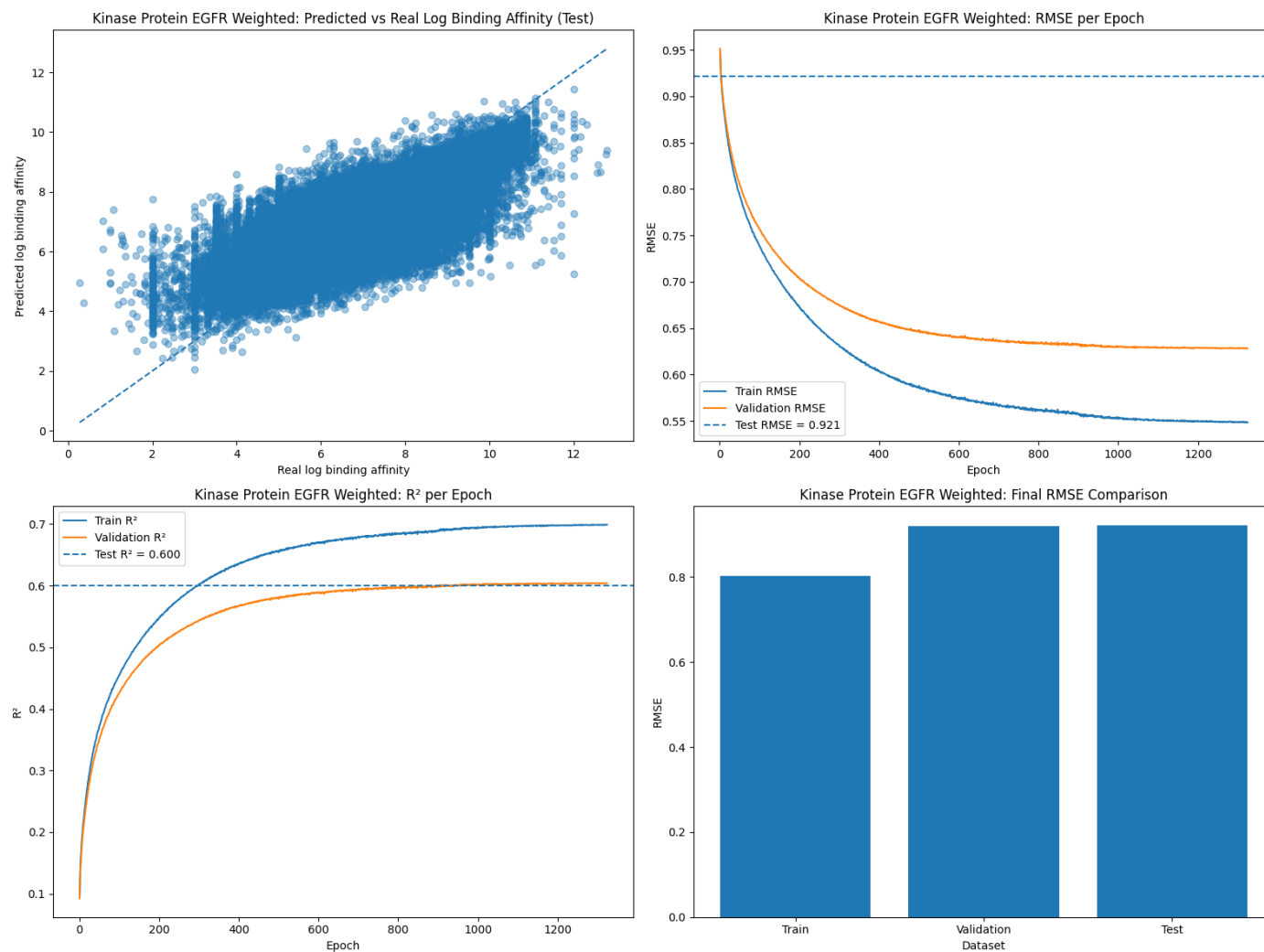


Figure A1: The Kinase ANN model indicates a decent predictive validation. The final Test R^2 scores is about 0.60, showing the model explains roughly 60% of the variability in the binding affinities. The training R^2 climbs up 0.7043 to while validation R^2 0.6071 stabilizes near indicating a good amount of modeling and understanding of molecular structure in the data. RMSE curves are crucial, as we used them to quantify prediction error during training. Training RMSE falls to 0.7957 while validation RMSE stops near 0.9158 and test RMSE stabilizes near 0.9189. These RMSE results indicate the ANN model generalizes over many different biochemical effects, opposed to memorization of other conditions in the model. This ANN is built on three hidden layers, using 224,112,56 neurons in a set of standardized and optimized input features, SGD derived gradient optimization, L2 regularization to stabilize weights and avoid overfitting. A learning rate of 0.0003, batch size of 256, and 1800 epochs of training were chosen to trade convergence speed for prediction performance over the long run in terms of protein affinity. Our model learned protein binding affinity based on linear sequence comparisons, however proteins are complex 3-D objects with interchangeable steric hindrance, dimerization, and so on. 60% accuracy is currently good enough to move along to family classification and finding patient mutation specific drug binding affinity.

Appendix B - (Checkpoint 2)

Checkpoint 2 Step 1 - Code Explanation

Clean protein sequences: The `clean_sequence()` function performs sequence preprocessing by first checking for null values and returning None for invalid inputs. Sequences are converted to uppercase strings and stripped of whitespace characters. The function then filters to retain only the 20 standard amino acids (ACDEFGHIKLMNPQRSTVWY), removing any non-standard characters that may represent unknown or modified residues. This ensures that all subsequent analyses operate on consistent, valid amino acid sequences. Empty sequences after cleaning are also rejected and assigned None values, which are later filtered from the dataset to prevent errors in feature calculations.

Extract amino acid composition: The `get_aa_composition()` function calculates the frequency of each amino acid by counting occurrences using Python's Counter class. For a given sequence, it determines the total length and computes the proportion of each amino acid by dividing its count by the total sequence length. The function returns a dictionary with 20 entries, one for each amino acid, where values sum to 1.0. This normalization makes composition features comparable across sequences of different lengths. If a sequence is empty, the function returns a dictionary of zeros to maintain consistent output dimensions.

Compute physicochemical properties: The `get_physicochemical_properties()` function iterates through each amino acid in the sequence and retrieves its predefined hydrophobicity, molecular weight, polarity, charge, and volume values from the `AA_PROPERTIES` dictionary. During iteration, it simultaneously counts residues falling into hydrophobic (positive hydrophobicity), polar (polarity=1), and charged (charge $\neq$ 0) categories. After processing all residues, it calculates mean values for hydrophobicity, molecular weight, polarity, and volume using NumPy's mean function. Net charge is computed by summing individual amino acid charges. Ratios of hydrophobic, polar, and charged residues are calculated relative to total sequence length. The function returns eight physicochemical metrics, returning NaN values for empty sequences to enable downstream filtering.

Calculate ratios: Ratios are computed within the `get_physicochemical_properties()` function automatically alongside other properties. The hydrophobic ratio counts residues with hydrophobicity values greater than zero and divides by total sequence length. The polar ratio counts residues with polarity equal to 1 (the polar designation) and divides by total. The charged ratio counts residues with charge values of -1 or +1 and divides by total. These ratios provide normalized measures of chemical character that are independent of protein length, making them suitable for comparing kinases of different sizes.

Add sequence length feature: The `get_sequence_length_features()` function simply returns the length of the cleaned sequence as a dictionary with key "seq_length". This feature is included separately from the ratio calculations to preserve information about kinase size, which varies across families and has biological relevance. Larger kinases may have additional domains or regulatory regions that influence drug binding behavior. The length feature also helps identify unusual variants and serves as a scaling factor for compositional features when needed.

Main processing pipeline: The `process_sequences()` function orchestrates feature extraction across the entire dataset. It first applies `clean_sequence()` to all sequences in the dataframe and filters out rows with None (invalid) sequences. It then identifies unique sequences to avoid redundant computation, storing sequences in a set. For each unique sequence, it calls `extract_all_features()` which combines length, composition, and physicochemical features into a single dictionary. Results are cached in a `feature_map` dictionary keyed by sequence string, enabling fast lookup for duplicate sequences. Finally, it iterates through all rows in the original dataframe, retrieving pre-computed features from the cache, and constructs a new dataframe by concatenating original columns with the new feature columns.

Checkpoint 2 Step 1 - Sanity Checks

No invalid sequences remain: The pipeline validates that sequence cleaning succeeded by checking that all sequences contain only the 20 standard amino acids. After applying `clean_sequence()`, the code filters the dataframe using `df["sequence_clean"].notna()` which removes any rows where cleaning returned None. This ensures that no sequences with invalid characters, unknown residues, or empty strings proceed to feature extraction. The code also prints the count of valid sequences to this stage, enabling manual verification that appropriate sequences remain after filtering.

All features are numeric: Feature extraction functions explicitly return numeric values for all features. Composition features are computed as frequencies (float values between 0.0 and 1.0). Physicochemical features return float values for averages and integer/float values for counts and charge properties. The `extract_all_features()` function combines all numeric features into a single dictionary, and the resulting Pandas dataframe automatically assigns appropriate dtypes. NaN values are intentionally returned for empty sequences, which will be handled as missing values in the dataframe. The code ensures no string or non-numeric data types are introduced into the feature columns, maintaining numerical consistency for downstream machine learning algorithms.

No missing values in key columns: While NaN values are intentionally returned for physicochemical features of empty sequences, the main processing pipeline ensures these rows are removed before feature assignment. When constructing the feature dataframe, only rows with valid clean sequences receive feature values. The final combined dataframe `result_df` will have no missing values in the newly created feature columns because all rows that pass the validity check yield complete feature vectors. Original columns from the input may contain missing values, but the newly engineered features (amino acid compositions, physicochemical properties, sequence length) will be complete for all retained rows. This can be verified by checking `result_df[feature_columns].isnull().sum().sum() == 0` after execution.

Output shape increases due to added features: The code validates that feature addition succeeded by printing the final dataframe shape after processing. The output dataset should have the same number of rows as the valid input sequences (after filtering) but 29 additional columns: 20 amino acid composition features, 8 physicochemical property features, and 1 sequence length feature, plus a `sequence_clean` column. If the input had N rows and M columns, the output should have N' rows ($N' \leq N$ after filtering) and $M+30$ columns (including `sequence_clean`). This dimensionality increase confirms that the feature engineering pipeline successfully expanded the feature space. The code also saves output to a timestamped CSV file with name `checkpoint_2_step_1_sequence_feature_data.csv`, enabling inspection of the final schema.

Checkpoint 2 Step 2 - Code Explanation

Clean sequences again for consistency: Step 2 re-applies the `clean_sequence()` function to ensure consistency with Step 1 preprocessing. Although the input to Step 2 comes from Step 1 output, which already contains cleaned sequences in the `sequence` column, the pipeline explicitly re-cleans and stores results in a `sequence_clean` column. This defensive programming practice ensures that if the pipeline is run independently or if intermediate files are modified, sequences are properly sanitized before embedding generation. Invalid sequences are filtered out just as in Step 1, maintaining data quality throughout the pipeline.

Generate amino acid embeddings: Rather than using external pre-trained models, this implementation employs a simplified but effective approach using `AA_EMBEDDINGS`, a dictionary containing 8-dimensional vectors for each of the 20 amino acids. These vectors are manually specified and inspired by BLOSUM62 substitution matrix principles, encoding evolutionary similarity between residues. Each amino acid receives a unique 8-element array where similar amino acids have numerically closer vectors. For example, alanine (A) and valine (V) likely share hydrophobic characteristics and would have similar vector values. These embeddings serve as the building blocks for sequence-level representations, allowing the code to aggregate meaningful chemical information without depending on large external language models.

Compute sequence-level statistics: The `get_sequence_embedding()` function calculates multiple statistical summaries across the sequence positions. It computes mean embeddings across all positions to capture average chemical character. Standard deviation embeddings capture variability in chemical properties across the sequence, highlighting regions of chemical heterogeneity. Maximum and minimum values identify extreme chemical characteristics that may indicate critical residues. These four statistical views (mean, std, max, min) are calculated on each of the 8 embedding dimensions, producing 32-dimensional summary vectors that together describe both central tendency and variability of chemical properties across the kinase sequence.

Apply position weighting: The `get_position_weights()` function generates position-specific weights that emphasize sequence termini, which are functionally important for kinase regulation and localization. For sequences shorter than `MAX_SEQ_LENGTH` (512), it applies a Gaussian-like weighting where positions near the ends receive higher weights. For longer sequences, it uses uniform weights. These weights are applied during the weighted embedding calculation in `get_sequence_embedding()`, where each amino acid’s embedding is multiplied by its position weight before averaging. This ensures that terminal residues (N-terminus and C-terminus) contribute disproportionately to the final representation, reflecting their biological importance in kinase function and drug binding.

Extract terminal features: The embedding function explicitly features N-terminal and C-terminal regions by computing separate embeddings for the first 10 and last 10 amino acids. The N-terminal embedding is calculated as the mean of embeddings for positions 0-9 (or fewer if sequence is shorter). The C-terminal embedding is calculated as the mean of embeddings for the last 10 positions. These terminal embeddings capture the distinct chemical character of regulatory regions frequently targeted by drugs and involved in oncogenic mutations. For EGFR, the C-terminal tail contains auto-inhibitory interactions that are disrupted in many cancer mutations, making this feature particularly relevant for detecting mutation-specific drug binding changes.

Combine all features into fixed-length vector (64-dimension): The final embedding is constructed by concatenating seven component vectors: mean embedding (8 dims), standard deviation embedding (8 dims), maximum embedding (8 dims), minimum embedding (8 dims), position-weighted embedding (8 dims), N-terminal embedding (8 dims), C-terminal embedding (8 dims), and normalized sequence length (1 dim). This produces a 65-dimensional vector,

which is padded with zeros or truncated to exactly 64 dimensions as specified by `EMBEDDING_DIM`. The use of truncation ensures that all sequences produce embeddings of identical dimensionality, enabling batch processing with machine learning algorithms that require fixed-size inputs. The multi-component design provides complementary views of sequence information from local (terminal) to global (statistical) perspectives.

Main processing pipeline: The `process_embeddings()` function mirrors the structure of Step 1's processing pipeline. It first cleans sequences and filters invalid rows. It identifies unique sequences for computational efficiency, then generates embeddings for each unique sequence using `get_sequence_embedding()`. Results are cached in an `embedding_map` dictionary keyed by sequence string. Finally, it iterates through all rows, retrieves embeddings from cache, constructs a dataframe with columns labeled `emb_0` through `emb_63`, and concatenates with the original dataframe. The output is saved to `checkpoint_2_step_2_deepseq_embedding.csv`, completing the sequence feature engineering pipeline with 64 additional deep learning-inspired features.

Checkpoint 2 Step 2 - Sanity Checks

All sequences produce embeddings: The pipeline validates embedding generation by processing all unique sequences and storing results in the `embedding_map` cache. The `get_sequence_embedding()` function includes defensive logic for edge cases: empty sequences return a zero vector, sequences shorter than N-terminal length (10 amino acids) fall back to mean embeddings for terminal features, and invalid characters are handled within the cleaning step. The code prints the count of valid sequences and unique sequences, allowing verification that embedding generation succeeded for all entries. The progress bar from `tqdm` indicates completion status for embedding generation across the dataset.

Embedding vectors have consistent length: The embedding function enforces fixed dimensionality by padding or truncating the concatenated vector to exactly `EMBEDDING_DIM` (64) dimensions. Inside `get_sequence_embedding()`, the code checks if `len(combined) < embedding_dim`: and pads with zeros using `np.pad()`, or `else`: truncates using slicing `combined[:embedding_dim]`. This ensures that regardless of sequence length, every embedding has exactly 64 values. When constructing the embedding dataframe, column names are generated programmatically as `f"emb_{i}"` for `i in range(embedding_df.shape[1])`, guaranteeing that all rows have the same columns. The output shape can be verified to confirm that all 64 embedding columns exist for every row.

No NaN or empty embeddings: The embedding design ensures robust handling of all input cases. Empty sequences return explicit zero vectors (`np.zeros(embedding_dim)`) rather than NaN values. Short sequences use fallback logic (mean embeddings) to populate terminal features rather than attempting operations that would produce NaN. The standard deviation calculation uses NumPy's `std` function, which returns 0.0 rather than NaN for length-1 arrays. Position-weighted averaging includes total weight checks to avoid division by zero. These defensive programming practices ensure that all returned embeddings contain valid numeric values. Combined with the filtering of invalid sequences during cleaning, the final embedding dataframe contains no NaN values in embedding columns, which can be verified with `result_df[[col for col in result_df.columns if col.startswith('emb_')]].isnull().sum().sum() == 0`.

Checkpoint 2 Step 3 - Local Alignment Diagram

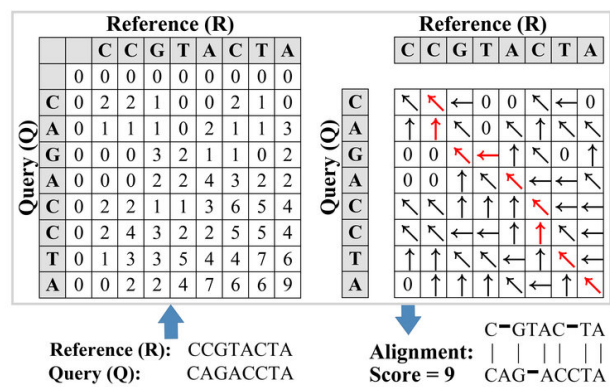


Figure B1: Illustration adapted from Liao et al., 2018 showing how local alignment scores are computed using matrix comparison of a reference sequence (R) and a query sequence (Q). The matrix on the left is the score matrix, showing each cell containing the optimal local alignment score calculated from match, mismatch, and gap penalties. The right matrix is the trace back matrix where red arrows are indicating the optimal alignment path used to generate the final alignment score.

Checkpoint 2 Step 3 - Kinase Family Dendrogram

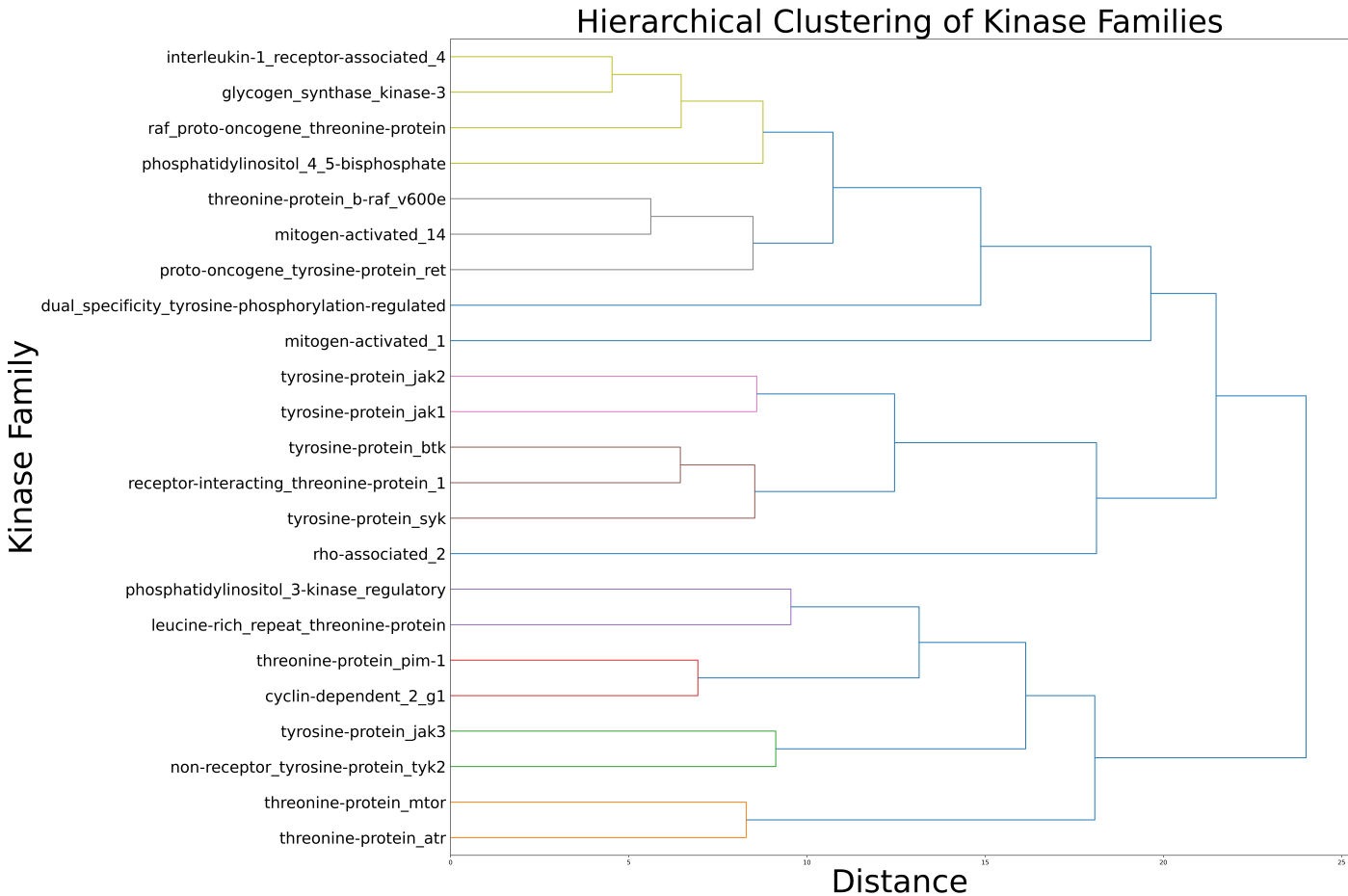


Figure B2: dendrogram visualization showing how kinase families group together using hierarchical clustering. Kinase families of similar type or biological function can be seen grouping together, whereas few kinase families stand out as unrelated or easily classifiable from others.

Checkpoint 2 Step 4 - Visualizations and Results

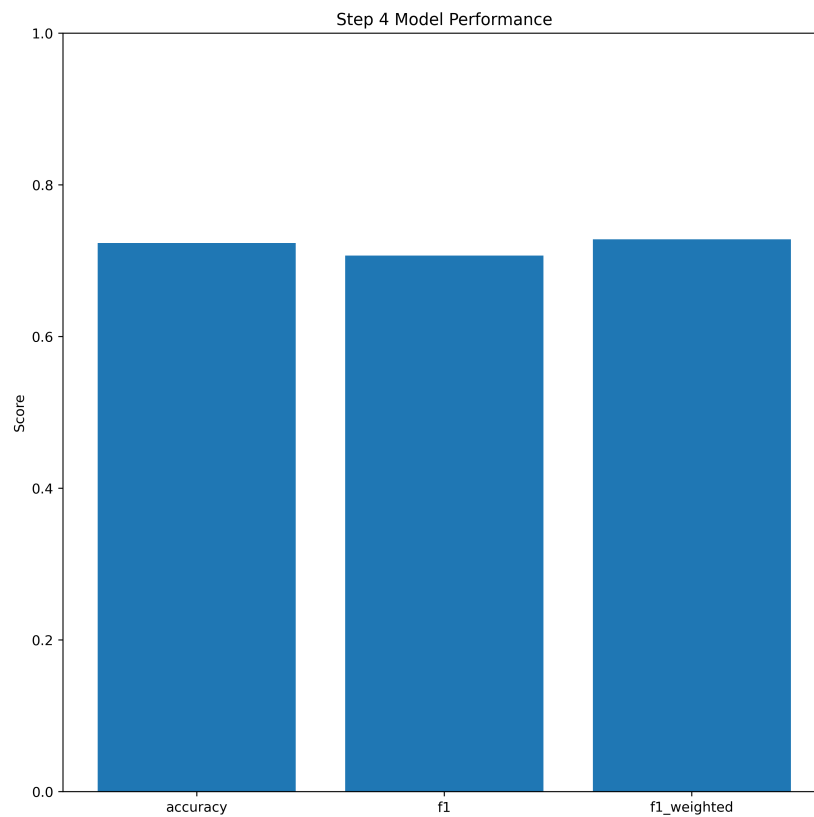


Figure B3: barplot showing results for XGBoost model metrics with consistent F1 macro (0.71) and F1 weighted (0.73) scoring and moderate accuracy performance. Train accuracy shows moderate overfitting with 90% accuracy compared to the test accuracy of 72%.

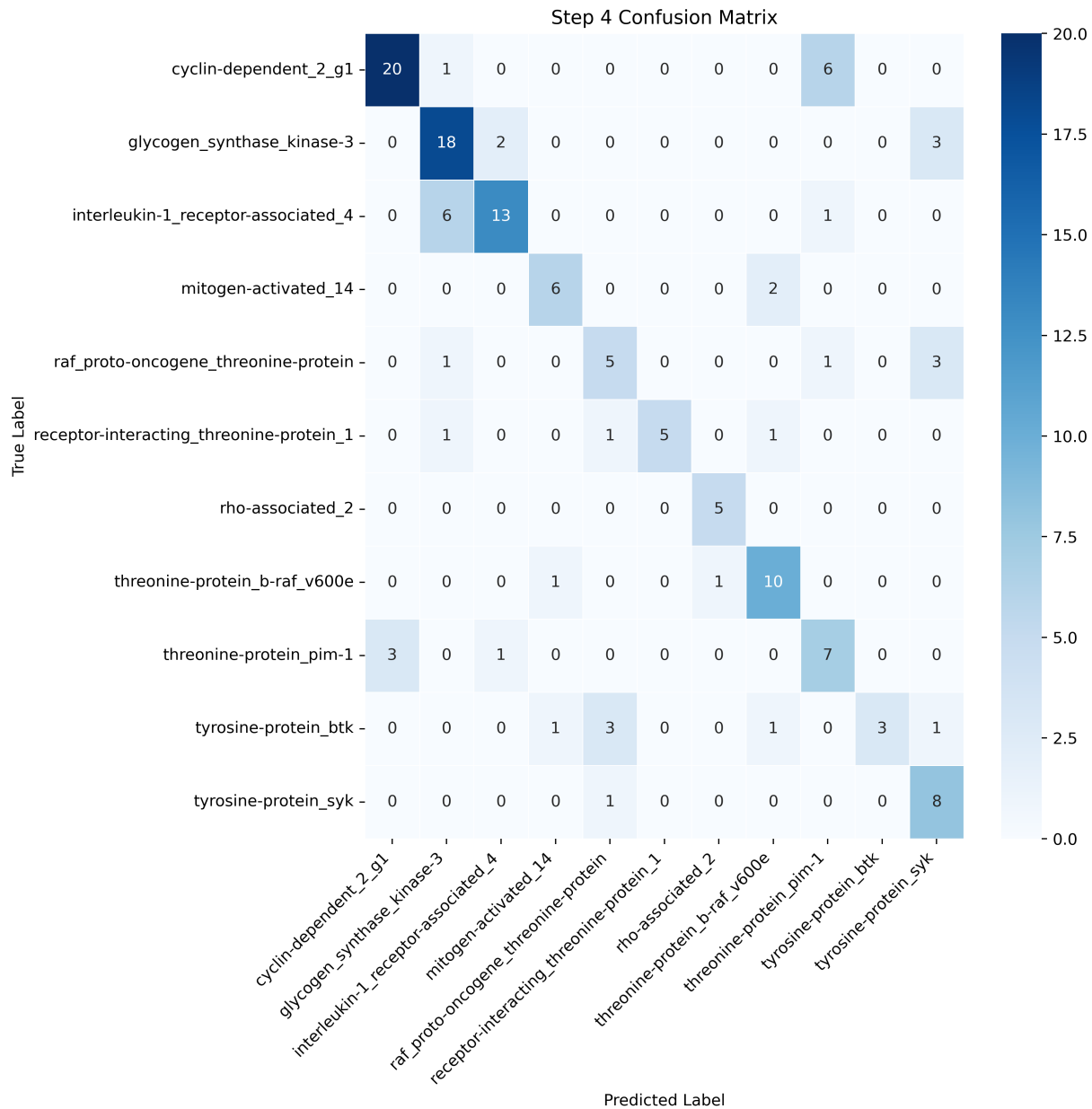


Figure B4: confusion matrix showing the classification results of the XGBoost model. The diagonal shows the correctly classified results for each kinase family, whereas results that are outside of the diagonal are misclassifications. The results of this matrix show that biological function overlap can cause issues with separating kinase families in classification tasks. The two most misclassified pairings are between glycogen_synthase_kinase-3 and interleukin-1_receptor-associated_4, which are both serine/threonine kinases that act in inflammation and immune response, as well as cyclin-dependent_2_g1 and threonine-protein_pim-1, which are both positive regulators of cell cycle progression. This supports that kinase families can vary significantly in structure, but biological function can cause overlapping in their classification assignments, making some families harder to separate than others.

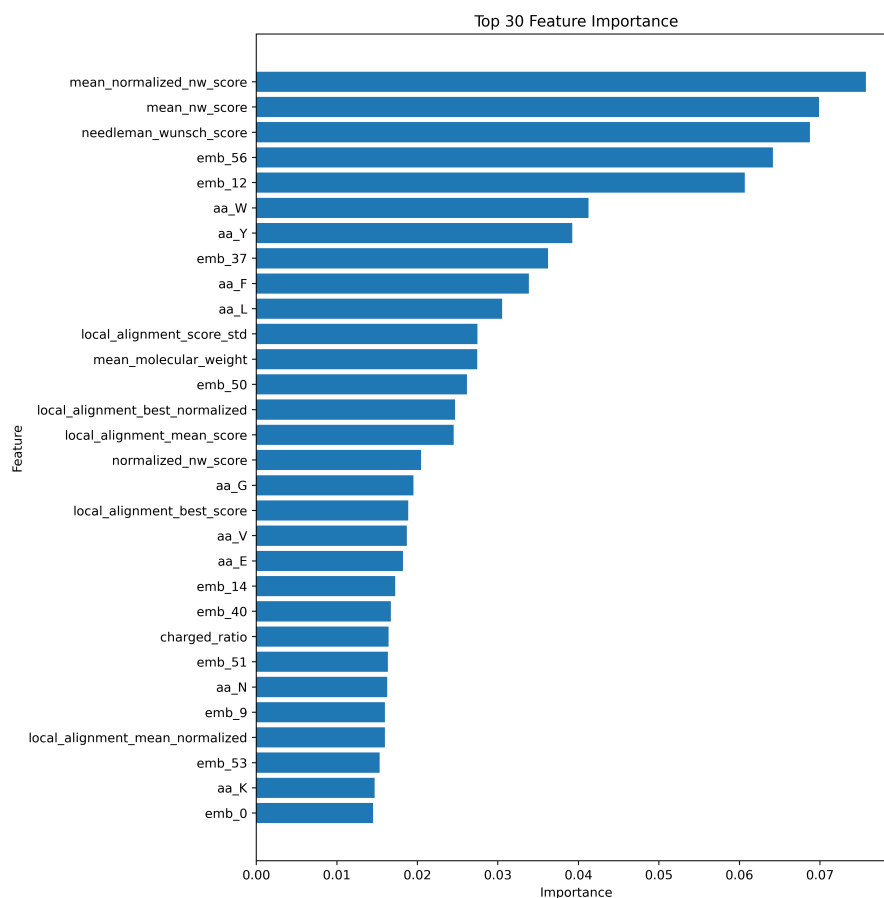


Figure B5: feature importance plot showing the features that are impacting the model predictions most, showing heavy reliance on the embeddings and amino acid compositions, as well as structural scores for global and local alignment. This supports that the wide range of features used in this project help to wholly depict our kinase families and their protein members.

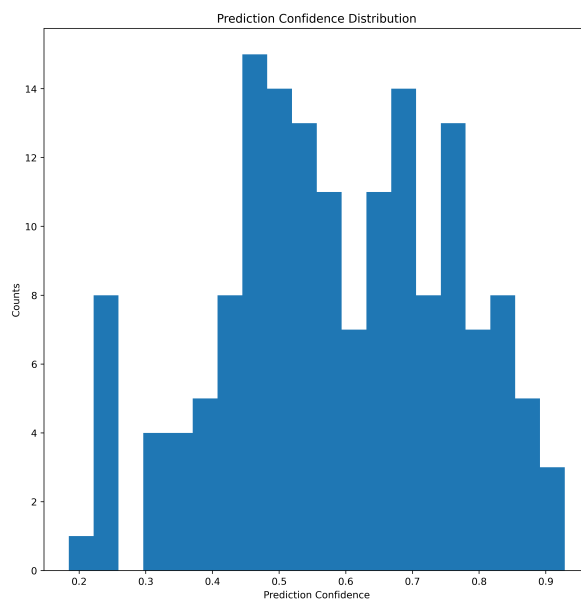


Figure B6: confidence histogram showing the prediction confidence of the model.

Appendix C - (Checkpoint 3)

	A	B	C	D	E	F	G	H	I
1	patient_number	ligand_bidingdb_number	ligand_short_name	ligand_iupac_name	target_name	ligand_smiles_string	ypred_binding_affinity	ytest_binding_affinity	logP_value
2	TCGA-55-A57B-01	37291	benzothiophene phenol analog	2-(4-hydroxy-3-propylphenyl)-1-benzothiophen-6-ol	EGFR / ERBB1 kinase	<chem>CCCC1cc(ccc1O)-c1cc2ccc(O)cc2s1</chem>	11.0602	11.2986	5.1272
3	TCGA-50-6591-01	37292	dihydroxy benzothiophene analog	2-(3,4-dihydroxyphenyl)-1-benzothiophen-6-ol	EGFR / ERBB1 kinase	<chem>Oc1ccc2cc(sc2c1)-c1ccc(O)c(O)c1</chem>	11.0419	11.2986	5.1272
4	TCGA-50-6591-01	37089	acetyl thienopyrimidine kinase inhibitor analog	N-[2-[4-[4-(trifluoromethyl)phenyl]pyrimidin-2-yl]oxy]-1-benzothiophen-3-yl]acetamide	EGFR / VEGFR	<chem>CC(=O)Nc1nc2c(Oc3cc(ncn3)-c3ccc(cc3)C(F)(F)F)cccc2s1</chem>	11.0201	10.9813	5.2106
5	TCGA-55-A57B-01	37086	propionyl thienopyrimidine kinase inhibitor analog	N-[2-[4-[4-(trifluoromethyl)phenyl]pyrimidin-2-yl]oxy]-1-benzothiophen-3-yl]propanamide	TRPV1	<chem>CCC(=O)Nc1nc2c(Oc3cc(ncn3)-c3ccc(cc3)C(F)(F)F)cccc2s1</chem>	10.9595	10.9726	5.8611
6	TCGA-50-6591-01	37284	raloxifene	[6-hydroxy-2-(4-hydroxyphenyl)-1-benzothiophen-3-yl]-[4-[2-(piperidin-1-yl)ethoxy]phenyl]methanone	Estrogen receptor / kinase-associated	<chem>Oc1ccc(cc1)-c1sc2cc(O)ccc2c1C(=O)c1ccc(OCCN2CCCC2)cc1</chem>	10.9245	10.9238	5.7806
7	TCGA-50-6591-01	37283	genistein	5,7-dihydroxy-3-(4-hydroxyphenyl)chromen-4-one	Protein tyrosine kinase / EGFR-associated	<chem>c1cc(ccc1C2=CC3C=C(C(C3C2=O)O)O)O</chem>	10.9073	10.7201	4.4564
8	TCGA-50-6591-01	618652	thiophene sulfonamide piperazine analog	N-[(4-chlorophenyl)methyl]-5-chloro-4-[3-(piperazin-1-yl)phenoxy]thiophene-2-sulfonamide	Kinase inhibitor scaffold	<chem>Clc1ccc(sc1Oc1cccc(c1)N1CCNCC1)S(=O)(=O)NCc1ccc(Cl)cc1</chem>	10.8635	10.9912	3.2191
9	TCGA-55-6980-01	37305	ethyl pyrazolopyrimidine kinase inhibitor analog	ethyl 7-ethyl-5-[[3-(methoxycarbonyl)-4-methylphenyl]amino]pyrazolo[1,5-a]pyrimidine-3-carboxylate	kinase receptor	<chem>CCOC(=O)c1cn2ncnc(Nc3cc(ccc3C)C(=O)NOC)c2c1CC</chem>	10.7080	10.6592	5.6365
10	TCGA-67-3770-01	27852	4-azidobenzenesulfonamide	4-azidobenzenesulfonamide	Carbonic anhydrase target	<chem>NS(=O)(=O)c1ccc(cc1)N=[N+]=[N-]CC(=O)OC[C@H]1O[C@H](OCc2cn(n2)-c2ccc(cc2)S(N)(=O)=O)[C@H](OC(C)=O)[C@@H](OC(C)=O)[C@H]1OC(C)=O</chem>	10.6870	10.7705	3.0800
11	TCGA-17-Z032-01	27853	peracetylated glucosyl triazole sulfonamide	[(2R,3R,4S,5R,6R)-3,4,5-triacetyloxy-6-[[4-(4-sulfamoylphenyl)triazol-1-yl]methoxy]oxan-2-yl)methyl acetate	Carbonic anhydrase target	<chem>CC(=O)OC[C@H]1O[C@H](OCc2cn(n2)-c2ccc(cc2)S(N)(=O)=O)[C@H](OC(C)=O)[C@@H](OC(C)=O)[C@H]1OC(C)=O</chem>	10.6866	10.7705	3.0800
12									

Figure C1: BindingDB provides experimentally derived protein ligand interaction data used for model evaluation. These small molecules are not marketed pharmaceuticals and the ligand numbers were from February 2026 BindingDB tsv file, thus we included ligand IUPAC names for future research purposes. Here y_{pred} ranges from 11.06 to 10.06 showing consistently strong predicted binding affinity across top ranked pairs. LogP values range from 5.6 to 3.08, which fall within a moderate drug like physicochemical space. These LogP ranges are acceptable for small molecules, and may support favorable membrane permeability for small molecules.

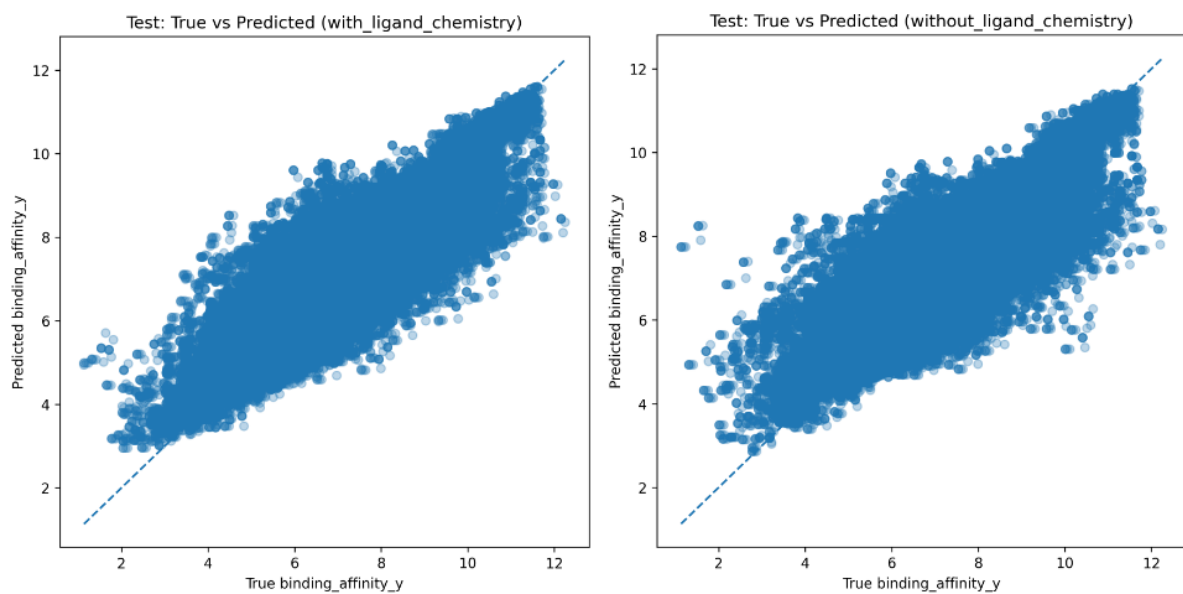
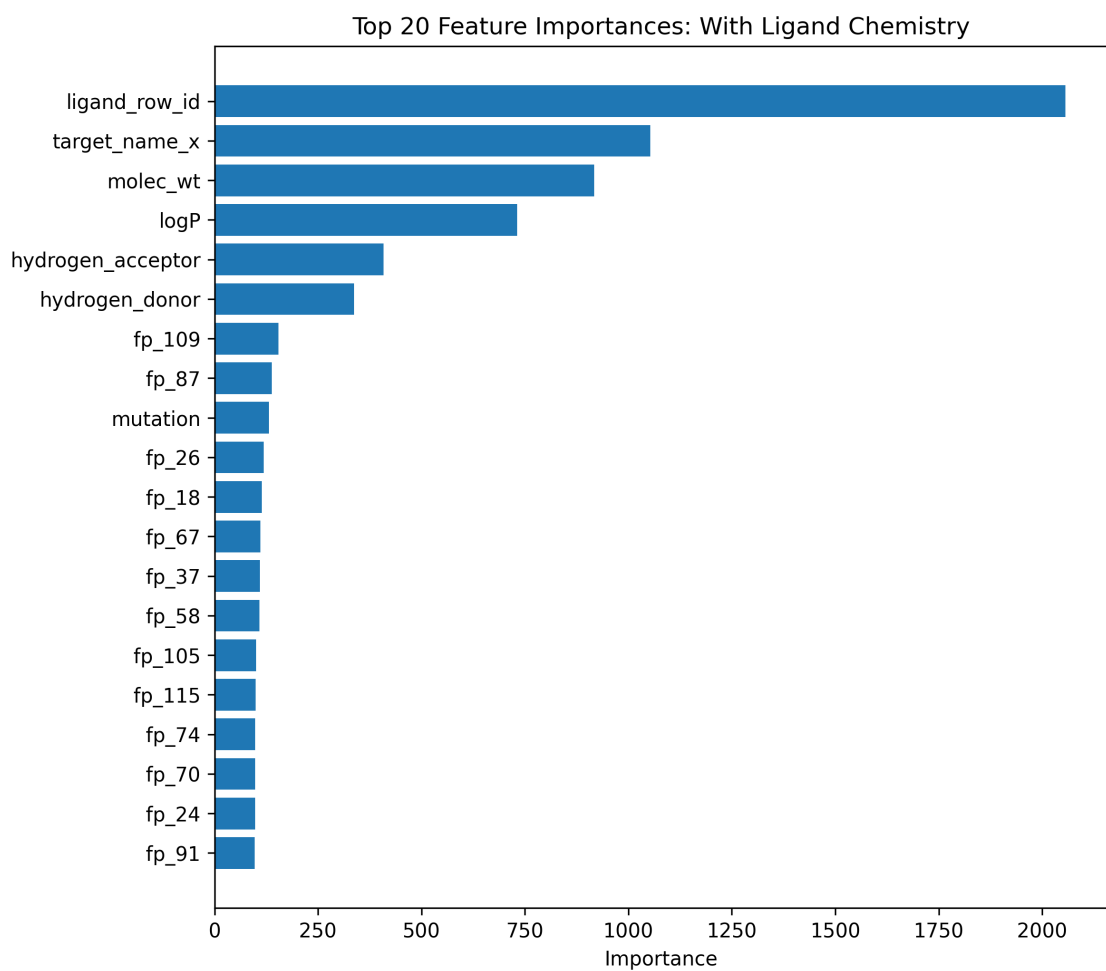


Figure C2 and C3: feature importance from ligand features in LightGBM and comparison to model with/without ligands.

Appendix D

	Percent CPU Used	Elapsed Time(m:ss)	Maximum Resident Set Size(GB)
ANN Model	386%	2h:15	3.25
XGBoost Model	183%	4:50	4.76
LightGBM Model	278%	0:58	3.31

Table D1: CPU Usage and Execution Time